

Applications of Transformer Networks in Bio- and Cheminformatics

Worksheet 5

Submission Deadline:	26. May 2025, 11:59 pm
Discussion of solutions:	03. June, 12:30 - 14:00

Submission Instructions:

- Upload a Jupyter notebook with your solutions to the exercises that you solved on your own PC. Upload the Python scripts and log files for any code executed on the HPC.
- If you are submitting multiple files, zip them together.
- Submit your solutions by uploading the Jupyter notebook to <https://uni-duesseldorf.sciebo.de/s/hCt1rTP23EeWmUC>. The uploaded file should have the following filename: "lastname_studentID_worksheet5".

Exercise 5.1 *Predicting K_M using small molecule representations* (30 Points)

The Michaelis constant K_M of an enzyme-substrate pair describes the affinity of an enzyme for its substrate. Formally, K_M is defined as the substrate concentration at which the enzyme operates at half its maximal speed. Although the K_M of an enzyme depends on both the enzyme and the substrates, the structure of the substrate already contains a lot of information about the K_M of the enzyme-substrate pair. In this exercise, we will use information about the substrate to predict the K_M value of an enzyme-substrate pair. For this task, you will need to download .csv files in *data/worksheet5*. The K_M values are stored in the columns named 'output'.

- (a) Compute extended-connectivity fingerprints (ECFPs), RDKit fingerprints, MACCS keys, and molecular descriptors for each SMILES string in the downloaded K_M datasets:
- To compute the molecular descriptors, use all 208 descriptors from the *rdkit.Chem* module *Descriptors*. If the resulting arrays contain values greater than 10^6 , replace them with zeros.
 - To compute ECFPs, you can use the function *GetMorganFingerprintAsBitVect* from the *rdkit.Chem* module *AllChem*. Set the radius to 2 and nBits to 1024.

- (b) For each of the four numerical small molecule representations computed in (a), perform a hyperparameter optimization of a gradient boosting model among the following parameters with the following upper and lower bounds:

- Number of trees ("n_estimators"): Lower bound: 30, Upper bound: 500,
- Maximum tree depth ("max_depth"): Lower bound: 2, Upper bound: 12,
- Learning rate: ("learning_rate"): Lower bound: 0.01, Upper bound: 0.5
- Min. child weight ("min_child_weight"): Lower bound: 1, Upper bound: 15,
- Lambda ("reg_lambda"): Lower bound: 0, Upper bound: 1,
- Alpha ("reg_alpha"): Lower bound: 0, Upper bound: 1.

Search for the best set of hyperparameters on the validation set by iterating over at least 50 different combinations of hyperparameters using the Python package *hyperopt*. Search for the set that leads to the highest coefficient of determination R^2 on the validation set.

- (c) For each small molecule representation, select the set of hyperparameters that yielded the best R^2 on the validation set in (b). Train a gradient boosting model with this set of hyperparameters on the training set and evaluate it on the test set, i.e., calculate MSE and R^2 for the test set.

Exercise 5.2 *Fine-Tuning ESM-2*

(70 Points)

In this exercise, we will fine-tune a ESM-2 model (model: esm2_t6_8M_UR50D) with 6 encoder layers for the task of predicting the Michealis constant K_M from only using enzyme sequence information in the model input. In the subparts of this exercise I will guide you through the necessary steps. I recommend training the model on the HPC if you cannot use a GPU on your own PC.

- (a) When fine-tuning the ESM-2 model, we want to build a feed-forward network (for K_M prediction) on top of the updated classification token. By default, the ESM-2 model includes a classification token in its input sequence. For the following tasks, you need to find where it is placed in the tokenized sequence. The function *alphabet.to_dict()* might help you.
- (b) Define a neural network that takes as input a protein amino acid sequence that is processed by the pre-trained ESM-2 model. After all input tokens are updated, the updated embedding of the classification token from layer 6 should be used as input for a fully connected neural network to predict the K_M of the protein.
- (c) Define a function to divide your training and validation data sets into batches of batch size 8.
- (d) Fine-tune the ESM-2 model for K_M prediction for at least 1 epoch on the training set. Set the loss function to mean squared error, the optimizer to the Adam optimizer, and the learning rate to 10^{-5} .
- (e) Validate how well this fine-tuned ESM-2 model performs on the validation set for predicting K_M : Calculate the MSE and the coefficient of determination R^2 .

- (f) Extract for all proteins (training, validation and test proteins) in the K_M data set the fine-tuned ESM-2 embeddings, i.e. the updated classification token embeddings from layer 6.
- (g) Perform hyperparameter optimization for a gradient boosting model as described in Exercise 4.2 using the new and fine-tuned ESM-2 embeddings.

Note on using the HPC: If you plan to use a GPU on the HPC for this task, please first check if there are enough GPUs available. If you are logged into the HPC via the terminal, you can check this with the following two commands:

```
module load hpc-tools
gpustat
```

Please use a GPU only if many GPUs are available, otherwise use a CPU. You can request a specific GPU, such as the RTX6000, by setting the following values in your bash script

```
#PBS -l select=1:ncpus=1:ngpus=1:mem=64GB:
accelerator_model=rtx6000
```

Exercise 5.3 *Use of LLMs*

(0 Points)

State for which exercise you have used LLMs (large language models) such as ChatGPT or GitHub Copilot. State which tools you have used and for which steps. This answer does not influence how many points you receive for your submission.